INSTRUCTOR COPY. Answers for the Intro to Python all-in-one booklet. Do not give to pupils. Starter and solution code are in `code/`.

Part 2 Console practice

Maths: `2 + 2` → 4; `8 / 2` → `4.0`; `7 * 6` → 42; `10 % 3` → 1; `2 + 3 * 4` → 14; `(2 + 3) * 4` → 20. Order matters: multiplication happens before addition unless brackets say otherwise.

Variables: `age = 16` shows nothing (it stores the value); `age` → 16; `score = 95` nothing; `score = score + 5` nothing (the new value replaces the old); `score` → 100; `name = "Zendaya"` nothing; `name` → `'Zendaya'`.

What does `=` mean? Store the value on the right in the name on the left. It is not "equals".

Types: `type(age)` → `<class 'int'>`; `type(name)` → `<class 'str'>`; `type(1.75)` → `<class 'float'>`; `type(True)` → `<class 'bool'>`.

Part 3 Functions

`add_one(5)` → 6. `add(3, 4)` → 7; `add(10, 20)` → 30.

Part 4 Function Ladder (code in code/ladder_solution.py)

```
def lucky():            # rung 1  -> 7
    return 7
def add_one(x):         # rung 2  -> add_one(5) is 6
    return x + 1
def hello():            # rung 3  -> nothing appears
    greeting = "hi"
def average(a, b):      # rung 4  -> 4.0
    return (a + b) / 2
def product(a, b):      # rung 4  -> 42
    return a * b
def add(a, b):          # rung 5
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    return a / b
def power(a, b):        # rung 7 (stretch) -> 1024
    return a ** b
```

Test chips: `lucky()` 7, `add_one(5)` 6, `average(3, 5)` 4.0, `product(6, 7)` 42, `add(2, 3)` 5, `subtract(10, 4)` 6, `multiply(6, 7)` 42, `divide(9, 2)` 4.5, `power(2, 10)` 1024.

Rung 3: `hello()` shows nothing. Without `return`, a function does its work but gives no value back to the console (it gives back `None`). **Rung 6:** `add(multiply(3, 4), 2)` → 14; `divide(add(6, 8), 2)` → `7.0`. Pupil's own rung 2 function: accept any `def` with one parameter and a `return` whose console value matches what they wrote on the chip.

Common mistakes: no `return`; misspelt function names (`substract`) which break the calculator; `return` not indented; `//` instead of `/` in `divide`; `print` instead of `return`; a body that is only a comment (`IndentationError`).

Part 5 Decisions

if/else: `temperature = 30` → `It's hot!`; `10` → `It's chilly`; `25` → `It's chilly` (25 is not greater than 25: good discussion point).

Comparisons: `==` equal to; `!=` not equal to; `>` greater than; `<` less than; `>=` greater than or equal to; `<=` less than or equal to.

elif: `72` → `B`; `95` → `A`; `40` → `Needs work`.

Teenager:

```
age = 16
if age >= 13 and age <= 19:
    print("You're a teenager!")
```

Part 6 Calculator (code/calculator_solution.py)

```
operation = input("Pick: + - * / ")
num1 = float(input("First number: "))
num2 = float(input("Second number: "))
if operation == "+":
    result = add(num1, num2)
elif operation == "-":
    result = subtract(num1, num2)
elif operation == "*":
    result = multiply(num1, num2)
elif operation == "/":
    result = divide(num1, num2)
print(f"Answer: {result}")
```

Tests: `*`, 6, 7 → `Answer: 42.0`; `/`, 20, 4 → `Answer: 5.0`; `-`, 9, 12 → `Answer: -3.0`. Any wording inside the `input` quotes is fine. Common faults: functions not pasted above the code; `"/"` written without quotes; `=` used instead of `==`.

Part 7 Loops and stretch

`for i in range(5)` prints `0 1 2 3 4`; the snacks loop prints `pizza chips soda`; the `while` loop prints `0 1 2`. Delete `count += 1` and it runs forever printing `0` (stop with the red square).

```
# S1: def power(a, b): return a ** b   and a new branch:  elif operation == "**": result = power
    (num1, num2)
# S2: def average(a, b): return (a + b) / 2   and a new elif for the chosen symbol
def ticket(age):                        # S3
    if age < 13:
        return 5
    else:
        return 10
for i in range(1, 11):                  # S4
    print(i * 7)
total = 0                               # S5  -> 5050
for n in range(1, 101):
    total += n
print(total)
```